

HelixLLM

HelixLLM

Un binaire unique, six modes — Inférence compatible OpenAI et Anthropic, de votre ordinateur portable à un cluster multi-hôte.

Résumé

HelixLLM est un système distribué d'inférence LLM de niveau entreprise conçu en Go : un binaire unique doté d'un système de modes permettant de passer d'un développement sur machine locale à une production multi-hôte. Il propose des API entièrement compatibles avec OpenAI et Anthropic via HTTP/3, une inférence locale avec llama.cpp, une chaîne de secours multi-fournisseurs avec notation, un pipeline RAG et un système d'agents ReAct.

Description courte

HelixLLM est un système distribué d'inférence LLM basé sur Go, fonctionnant avec un seul binaire. Il expose des API compatibles avec OpenAI et Anthropic via HTTP/3, exécute une inférence locale avec llama.cpp, découvre et note automatiquement des fournisseurs cloud gratuits pour former une chaîne de secours, et intègre un pipeline de connaissances RAG ainsi qu'un agent ReAct capable d'appeler des outils — le tout déployable selon six modes.

Description détaillée

HelixLLM est un système distribué d'inférence LLM de niveau entreprise, développé en Go avec Gin. Son atout principal réside dans le fait qu'un seul artefact couvre tous les niveaux d'échelle. Il se compile en un binaire unique dont le système de modes détermine, au moment du déploiement, ce que ce binaire *devient* : exécuté en mode `full`, il fonctionne comme une instance tout-en-un sur un ordinateur portable, ou bien répartit ses responsabilités entre les modes `gateway`, `brain`, `knowledge`, `agents` et `control` sur

plusieurs hôtes — le même code, simplement réorganisé plutôt que réécrit, de la machine d'un développeur à un cluster de production.

Il maîtrise deux dialectes à la perfection : des API entièrement compatibles avec OpenAI et Anthropic, permettant aux clients SDK existants des deux écosystèmes de fonctionner sans modification, le tout servi via HTTP/3 (QUIC) avec un basculement automatique vers HTTP/2 et TLS 1.3. L'inférence locale s'effectue via llama.cpp avec prise en charge de CUDA, Metal et ROCm, ce qui permet au même build d'accélérer sur les matériels Nvidia, Apple et AMD. Son point fort réside dans la chaîne de secours multi-fournisseurs, qui transforme l'instabilité notoire des services d'inférence cloud gratuits en une ressource gérée et auto-réparante : HelixLLM découvre automatiquement des modèles gratuits auprès de plus de 7 fournisseurs cloud (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), les évalue via LLMsVerifier toutes les 5 minutes, et achemine les requêtes à travers la chaîne classée avec un basculement automatique en cas d'erreurs 429/5xx — le tout en garantissant toujours llama.cpp local comme solution de dernier recours, afin qu'une requête ne soit jamais simplement rejetée faute de fournisseur disponible.

Au-delà de l'inférence brute, HelixLLM constitue une plateforme applicative complète : un pipeline de connaissances RAG (ingestion, découpage, embedding, recherche vector) et un système d'agents ReAct avec appel d'outils, gestion de sessions de conversation et intégration RAG sont inclus dans le même binaire. Le système de modes se révèle également avantageux au niveau réseau — en mode full, toutes les couches communiquent via des appels Go directs en processus, sans surcharge réseau, tandis que le même binaire, réparti sur plusieurs hôtes, se coordonne via gRPC, SSE et Kafka. Pour parfaire le tout, on trouve une négociation de contenu Brotli/gzip, un streaming SSE reproduisant à l'identique les formats OpenAI et Anthropic, une authentification par clé API et JWT avec limitation de débit, des métriques Prometheus, un traçage OpenTelemetry, ainsi qu'un large éventail de sous-modules Go dédiés à l'infrastructure de production.

Contenu

Pourquoi nous l'avons conçu

Les équipes ont besoin d'une inférence portable, compatible avec les standards et résiliente — sans avoir à réécrire les clients ni dépendre d'un seul fournisseur ou d'une seule machine. HelixLLM a été conçu pour qu'un même binaire puisse s'exécuter en local

pour le développement et s'adapter à un cluster de production multi-hôtes, en utilisant les dialectes OpenAI et Anthropic déjà exploités par les clients.

Pourquoi c'est révolutionnaire

Il condense toute une pile d'inférence — passerelle, inférence locale, repli cloud, RAG et agents — en un seul binaire contrôlé par un sélecteur de mode, permettant ainsi de choisir l'architecture au moment de l'exécution plutôt que de lancer un projet de migration. Et il transforme ce qui était autrefois un point faible en atout : la fiabilité des fournisseurs cloud devient une préoccupation centrale, mesurée en continu, gérée par une chaîne de repli auto-réparante et notée, qui réévalue les fournisseurs toutes les quelques minutes et bascule systématiquement vers une inférence locale garantie. Ce que cela permet, c'est un point de terminaison unique sur lequel on peut vraiment compter — compatible avec les standards, portable du portable au cluster, et incapable de tomber en panne parce qu'un fournisseur en amont a limité le débit ou échoué.

Ce qui est innovant

- Un binaire unique doté d'un système à six modes, capable de fonctionner en mode tout-en-un ou en rôles distribués — appels Go directs en mode `full`, gRPC/SSE/Kafka en mode distribué — de sorte que la topologie de déploiement évolue sans modification du code ni surcoût réseau imprévu.
- Une chaîne de repli multi-fournisseurs auto-découvrante et notée, couvrant plus de 7 fournisseurs gratuits, classés en continu par LLMsVerifier avec bascule automatique en cas d'erreurs 429/5xx et recours garanti à llama.cpp en dernier ressort — transformant la capacité des offres gratuites en capacité fiable.
- Des interfaces compatibles à la fois avec OpenAI et Anthropic, servies via HTTP/3 avec repli automatique sur HTTP/2, permettant aux clients des deux écosystèmes de se connecter sans modification.
- Une inférence locale couvrant CUDA, Metal et ROCm à partir d'une seule base de code — le même build s'exécute en accéléré sur du matériel Nvidia, Apple et AMD.

Principaux défis techniques et nos solutions

- **Passer d'un hôte à plusieurs sans réécriture.** La plupart des systèmes imposent une frontière stricte entre

« développement local » et « production distribuée », et la franchir implique une refonte architecturale. Nous avons effacé cette frontière grâce à un système de modes intégré à un binaire unique : les mêmes couches communiquent via des appels directs en mode full et basculent de manière transparente vers gRPC/SSE/Kafka en modes distribués, si bien que le passage à l'échelle se résume à un changement de configuration plutôt qu'à une migration.

- **Fournisseurs cloud gratuits, peu fiables et limitant le débit.** L'inférence en offre gratuite est rapide... jusqu'à ce qu'elle renvoie une erreur 429 ou disparaisse en cours de requête. Nous l'avons rendue fiable en découvrant automatiquement les modèles disponibles, en les notant avec LLMsVerifier, en surveillant proactivement les en-têtes de limitation de débit pour éviter les fournisseurs sur le point de brider les requêtes, et en basculant automatiquement vers le modèle suivant dans la chaîne classée, jusqu'à llama.cpp en local — de sorte que l'instabilité du pool ne touche jamais l'appelant.
- **Compatibilité des clients entre deux écosystèmes.** Réécrire les clients pour adopter un nouveau backend d'inférence est inenvisageable. Nous avons implémenté à la fois les formats OpenAI et Anthropic — y compris leurs formats de streaming SSE distincts — afin que les SDK des deux camps puissent pointer vers HelixLLM et fonctionner sans modification.

Contenu

Pile technologique

- **Go + Gin** — choisis car un runtime monolithique axé sur la concurrence est ce qui rend possible l'ensemble du système de modes : une seule compilation pouvant servir de serveur local ou de nœud de cluster. Il embarque l'intégralité du système ainsi que la couche HTTP du gateway.
- **HTTP/3 (QUIC) + TLS 1.3, avec repli sur HTTP/2** — retenus pour leur transport moderne, à faible latence et résilient aux déconnexions, exposé en tant que surface serveur avec négociation automatique, permettant aux clients incapables de gérer QUIC de basculer discrètement vers HTTP/2.
- **llama.cpp (CUDA/Metal/ROCm)** — choisi pour son inférence locale portable, accélérée sur les backends Nvidia, Apple et AMD à partir d'une seule base de code ; il sert également de fournisseur de dernier recours garanti, évitant ainsi à la chaîne de repli de jamais atteindre une impasse.
- **LLMsVerifier** — adopté pour transformer la question « *quel fournisseur est performant en ce moment ?* » en un chiffre ; il évalue et classe la chaîne de repli cloud toutes les 5 minutes,

de sorte que le routage s'adapte à la qualité en temps réel plutôt qu'à des hypothèses obsolètes.

- **Fournisseurs cloud (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together)** — sélectionnés pour exploiter les capacités gratuites de plusieurs sources en amont ; découverts et classés automatiquement en une seule chaîne de basculement, aucun fournisseur ne constituant un point de défaillance unique.
- **gRPC + SSE + Kafka** — retenus comme protocoles de transport inter-modes pour les déploiements distribués : gRPC pour les appels service à service, SSE pour le streaming, et Kafka pour le flux d'événements découplé entre les rôles.
- **Magasin de vecteurs / embeddings** — choisi pour alimenter le pipeline de connaissances RAG de bout en bout : ingestion, découpage, vectorisation et recherche dans les documents qui ancrent les réponses du modèle.
- **Prometheus + OpenTelemetry** — adoptés pour le suivi des métriques et le traçage distribué accompagnant une requête à travers tous les modes déployés.
- **Sous-modules vasic-digital Go** — choisis pour réutiliser des primitives d'infrastructure de production éprouvées plutôt que de les reconstruire, garantissant ainsi une cohérence de la base du système avec l'ensemble de la pile.

Statut et notes de transparence

- **Statut : bêta.** Système d'inférence distribuée fonctionnel et en développement actif.
- **Licence : à déterminer.** Le dépôt ne déclare aucune licence dans ses métadonnées (licenseInfo null) — cette information est NON VÉRIFIÉE et doit être clarifiée avant toute mention d'une licence.
- Le dépôt canonique pointe actuellement vers github.com/HelixDevelopment/llm ; le chemin HelixLLM y redirige. Les seuils de couverture et le nombre de sous-modules indiqués dans le README sont auto-déclarés.

Niveau de priorité : Helix-primaire.